

Curve Ellittiche

Dalla Teoria alle Applicazioni IoT

Applebee Kelly

Maggio 2026

Chapter 1

Introduzione

La necessità di trasmettere informazioni riservate su canali di comunicazione pubblici ha reso la crittografia a chiave pubblica uno degli strumenti fondamentali dell'informatica moderna. Tra le costruzioni crittografiche asimmetriche, la crittografia su curve ellittiche (ECC) si distingue per l'elevata sicurezza ottenibile con chiavi di dimensioni ridotte, caratteristica che la rende particolarmente adatta ai sistemi con risorse computazionali limitate, tra cui i dispositivi IoT.

La cifratura a chiave pubblica è l'evoluzione della cifratura a chiave privata, quest'ultima infatti presentava una sola chiave, simmetrica, nota sia al mittente che al destinatario, comunicata senza utilizzare canali non protetti (di solito comunicata di persona evitando dispositivi connessi ad internet) in modo da poter successivamente instaurare una comunicazione su un canale sicuro, dove sia il mittente che il destinatario dispongono della stessa chiave.

La cifratura a chiave pubblica introduce una nuova chiave per ogni utente all'interno della comunicazione, una chiave pubblica usata per cifrare il messaggio, e una privata usata per decifrare il messaggio ricevuto; la chiave pubblica di cifrazione è nota a tutti gli utenti interessati a comunicare con una certa persona, la seconda chiave (di decifrazione) è privata e nota solamente ad un individuo, una volta ricevuto il messaggio cifrato con la chiave pubblica del destinatario, questo usa la sua chiave privata per decifrare il messaggio.

Uno dei diversi modelli di attacco crittografico è noto come *known-plaintext* [1].

Si consideri il seguente scenario: due utenti vogliono comunicare attraverso un canale non sicuro, perciò utilizzano un cifrario per nascondere i messaggi inviati.

Nonostante le precauzioni consideriamo l'evento in cui un attaccante sia stato

in grado di accedere a delle coppie $(P_i, C_i), i = 1, \dots, N$ di *plaintext*¹ noti, e i crittogrammi ad essi associati. Date abbastanza coppie diventa triviale risalire al messaggio originale dato un nuovo crittogramma non visto prima di allora.

Un esempio di known-plaintext attack è *probable word*

RSA e Diffie-Hellman, se implementati correttamente e con parametri adeguati, sono ancora considerati sicuri contro gli attacchi pratici noti.

1.1 Campi

Prima di parlare di algoritmi crittografici è necessario introdurre alcuni concetti matematici, primo tra i quali i *campi*.

Definizione 1. *Un campo $(K, +, \cdot)$ è un insieme non vuoto che presenta operazioni binarie le quali soddisfano le seguenti proprietà:*

- $(K, +)$ è un gruppo con elemento neutro 0
- $(K \setminus \{0\}, \cdot)$ è un gruppo abeliano² con elemento neutro 1
- La moltiplicazione è distributiva rispetto all'addizione: $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$

In altre parole si vuole che ogni elemento, escluso lo 0 abbia un inverso moltiplicativo.

1.1.1 Esempi di campi

- L'insieme dei numeri razionali $\mathbb{Q}$ è un campo.
- L'insieme dei numeri reali $\mathbb{R}$ è un campo
- L'insieme $\mathbb{Z}/\mathbb{Z}_p$ delle classi di resto modulo p è un campo se e solo se p è un numero primo

¹con plaintext si intende un messaggio non cifrato

²Un gruppo abeliano è definito come un gruppo la cui operazione binaria interna gode della proprietà commutativa, ossia dato un gruppo $(G, \cdot)$, questo è abeliano se

$$a \cdot b = b \cdot a, \forall a, b \in G. \quad (1.1)$$

1.2 Funzione di Eulero

Definizione 2. La funzione di Eulero può essere scritta come $\Phi(n) = |\mathbb{Z}_n^*|$

Questa funzione risulta utile perché dato:

- $\Phi(p) = p - 1$
- $\Phi(pq) = (p - 1)(q - 1)$ ³
- a primo con b allora $\Phi(ab) = \Phi(a)\Phi(b)$
- $\Phi(n)$ difficile da calcolare senza conoscere la fattorizzazione di n

Definizione 3. Per ogni $p > 1$ primo e per ogni $k \geq 1$ intero:

$$\Phi(p^k) = \Phi(p) \cdot p^{k-1} \quad (1.2)$$

$$= (p - 1) \cdot p^{k-1} \quad (1.3)$$

1.3 RSA

Il cifrario RSA fonda la sua sicurezza sulla difficoltà di fattorizzazione di grandi numeri interi, il che lo rende sostanzialmente inviolabile per chiavi adeguatamente grandi

1.3.1 Creazione delle chiavi

Un Destinatario deve compiere queste operazioni in modo da creare le chiavi pubbliche e private:

1. scegliere due numeri primi molto grandi p, q
2. calcolare $n = p \cdot q$ e $\Phi(n) = (p - 1) \cdot (q - 1)$
3. scegliere un numero intero e minore di $\Phi(n)$ e primo con esso
4. calcolare l'intero d inverso di e modulo $\Phi(n)$
5. rendere pubblica la chiave $k_{\text{pub}} = (e, n)$, e mantenere segreta la chiave $k_{\text{prv}} = d$

³Valido solo quando p e q sono coprimi

Per codificare correttamente il messaggio questo deve essere trasformato in binario, il quale viene trattato come un numero intero m , la lunghezza del quale deve risultare $< n$, il che è possibile dividendo il messaggio m in blocchi di al più $\lfloor \log_2(n) \rfloor$ bit

$$c = m^e \bmod n \quad (\text{cifatura}) \quad (1.4)$$

$$m = c^d \bmod n \quad (\text{decifatura}) \quad (1.5)$$

1.4 Diffie-Hellman

A differenza di RSA Diffie-Hellman richiede una collaborazione da parte dei due utenti per la generazione delle chiavi:

1. L'utente A sceglie un numero primo p , un generatore g di $\mathbb{Z}_p^*$ e un intero $a > 0$
2. A calcola $A = g^a \bmod p$ e spedisce (g, p, A) a B
3. L'utente B sceglie un intero $b > 0$, calcola $B = g^b \bmod p$ e lo spedisce ad A
4. B calcola la chiave segreta $K = A^b \bmod p$
5. A calcola la chiave segreta $K = B^a \bmod p$

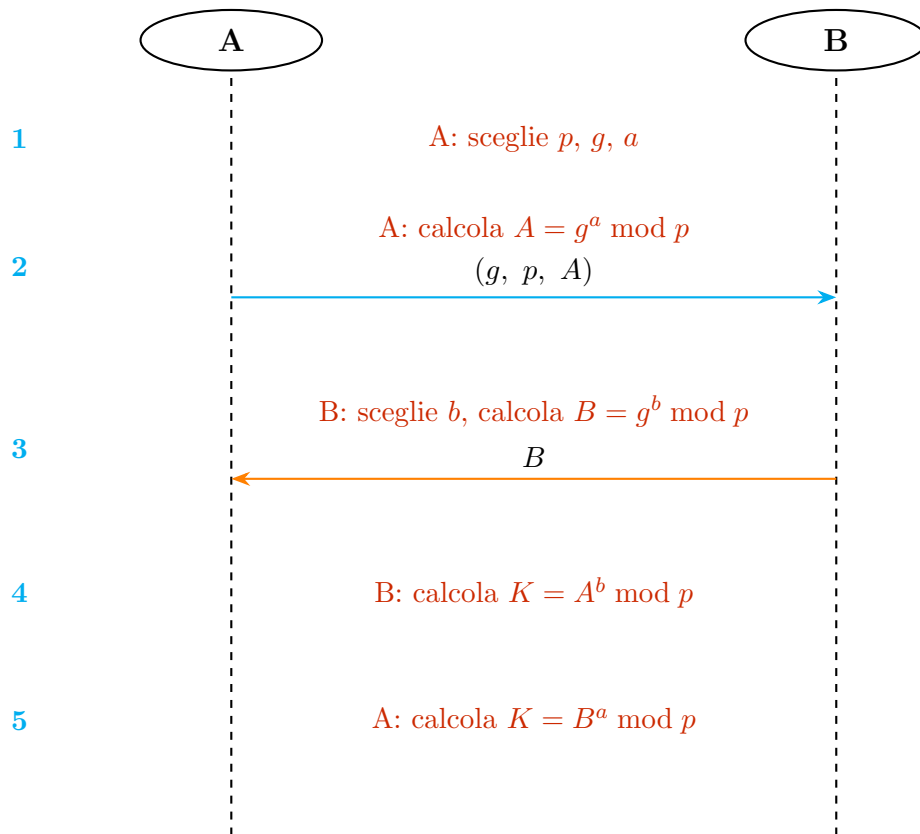


Figure 1.1: Scambio di chiavi Diffie-Hellman.

La sicurezza di questo protocollo è data dal fatto che anche se un crittoanalista potrebbe aver intercettato i valori p, g, A, B scambiati in chiaro tra i due utenti, questo dovrebbe risolvere l'equazione $A = g^a \bmod p$ rispetto ad a oppure $B = g^b \bmod p$ rispetto a b ovvero calcolare il logaritmo discreto di A , o di B 2.2.3.

Chapter 2

Curve Ellittiche

2.1 Curve ellittiche su $\mathbb{R}$

Una curva ellittica è una curva piana definita da un'equazione del tipo:

$$y^2 = x^3 + ax + b^1 \quad (2.1)$$

Questa possiede un'operazione di somma fra punti appartenenti alla curva, rispetto alla quale essa è un gruppo abeliano il cui elemento neutro è costituito dal punto all'infinito $\mathcal{O}$ 2.1.2.

Una curva ellittica è caratterizzata dal fatto che sommando due punti se ne ottiene un terzo appartenente alla stessa curva.

Nella definizione di curva ellittica viene definito anche il *punto all'infinito* $\mathcal{O}$ intuitivamente pensato come il terzo punto di intersezione di rette parallele all'asse y ; la definizione rigorosa segue dalle coordinate omogenee 2.1.2 [3].

¹Questo funziona solamente quando la caratteristica del campo K è diversa da 2 e 3, altrimenti l'equazione diventerebbe $y^2 + axy + by = x^3 + cx^2 + dx + e$ dove $a, b, c, d, e \in K$

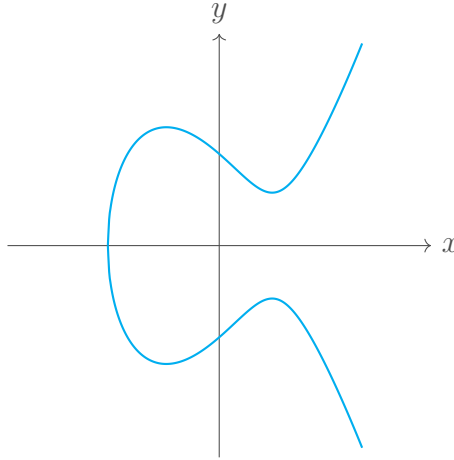


Figure 2.1: Rappresentazione di una curva ellittica

2.1.1 Caratteristiche curve ellittiche

1. Chiusura:

$$\forall P, Q \in E(a, b) : P + Q \in E(a, b) \quad (2.2)$$

2. Elemento neutro:

$$\forall P \in E(a, b) : P + \mathcal{O} = \mathcal{O} + P = P \quad (2.3)$$

(vedere Somma su curve ellittiche)

3. Associatività:

$$\forall P, Q, R \in E(a, b) : P + (Q + R) = (P + Q) + R \quad (2.4)$$

4. Commutatività:

$$\forall P, Q \in E(a, b) : P + Q = Q + P \quad (2.5)$$

2.1.2 Punto all'infinito

Definizione 4. *Punto improprio definito in Geometria proiettiva associato a una direzione nel piano: ogni famiglia di rette parallele converge in un unico punto all'infinito, l'insieme di questi punti forma la retta impropria.*

Nelle curve ellittiche, il punto all'infinito $\mathcal{O}$ è unico e funge da elemento neutro dell'addizione del gruppo.

Intuizione geometrica

I punti all'infinito corrispondono ai punti $[X : Y : Z]$ con $Z = 0$. Passando alle coordinate omogenee $[X : Y : Z]$ con $x = X/Z$, $y = Y/Z$, l'equazione di Weierstrass diventa:

$$Y^2Z = X^3 + aXZ^2 + bZ^3 \quad (2.6)$$

Ponendo $Z = 0$ si ottiene $X = 0$, da cui il punto all'infinito è $\mathcal{O} = [0 : 1 : 0]$ [3].

2.1.3 Somma su curve ellittiche

Definizione 5. *Ogni retta interseca una curva ellittica in al più tre punti [2]*

A questo si aggiunge un secondo fatto, non implicato dalla definizione precedente: se una retta interseca una curva $E(a, b)$ in due punti distinti P e Q , allora interseca sempre anche un terzo punto R .

Se la retta è invece verticale, oltre ai punti P e Q essa interseca anche il punto all'infinito $\mathcal{O}$, che viene considerato come terzo punto di intersezione. Dati tre punti P , Q , R di una curva ellittica $E(a, b)$ se P , Q e R sono disposti su una retta, allora la loro somma si pone uguale al punto all'infinito:

$$P + Q + R = \mathcal{O} \quad (2.7)$$

Da queste definizioni è possibile ricavare la regola per sommare due punti P e Q

- Si consideri la retta passante per i punti P e Q , oppure la tangente alla curva in P nel caso P e Q coincidano.
- Si determina il punto di intersezione R tra la curva e la retta appena tracciata
- Si definisce la somma come $P + Q = -R$ dove $-R$ rappresenta il punto simmetrico a R rispetto all'asse delle ascisse

Da un punto di vista pratico questo vuol dire:

Considerando P e Q distinti e $P \neq -Q$

1. $P = (x_p, y_p), Q = (x_q, y_q),$
2. $P + Q = S = (x_s, y_s)$
3. $\lambda = \frac{y_q - y_p}{x_q - x_p}$

$$4. x_s = \lambda^2 - x_p - x_q$$

$$5. y_s = -y_p + \lambda(x_p - x_s)$$

Caso 1 – Elemento neutro (punti con la stessa ascissa). Se P e Q hanno la stessa coordinata x , allora necessariamente $Q = -P$ (sono simmetrici rispetto all'asse delle ascisse, essendo entrambi soluzioni di $y^2 = x^3 + ax + b$ per lo stesso x). La retta che li congiunge è quindi verticale e, come visto in precedenza, interseca la curva anche nel punto all'infinito $\mathcal{O}$, che diventa il terzo punto di intersezione $R = \mathcal{O}$. Applicando la regola generale $P + Q = -R$ e osservando che $-\mathcal{O} = \mathcal{O}$ (essendo l'elemento neutro), si ottiene:

$$P + Q = P + (-P) = \mathcal{O} \quad (2.8)$$

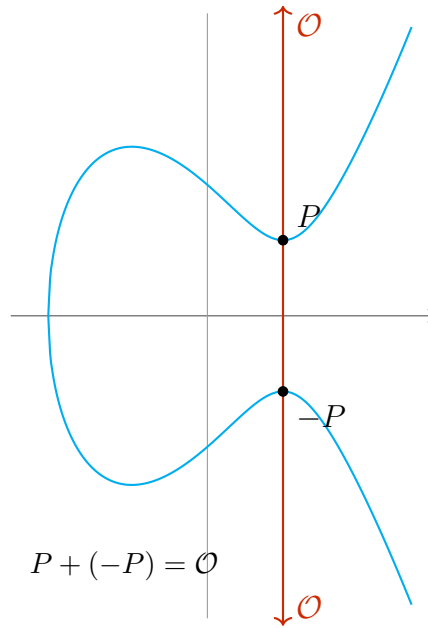


Figure 2.2: Somma di punti con la stessa ascissa

Caso 2 – Raddoppio di un punto ($P = Q$). Quando i due punti da sommare coincidono, la retta secante degenera nella tangente alla curva in P , come già anticipato. Da un punto di vista pratico:

$$1. P = (x_p, y_p), 2P = S = (x_s, y_s)$$

$$2. y_p \neq 0 \Rightarrow \lambda = \frac{3x_p^2 + a}{2y_p}$$

3. $y_p = 0 \Rightarrow$ tangente verticale $\Rightarrow 2P = \mathcal{O}$

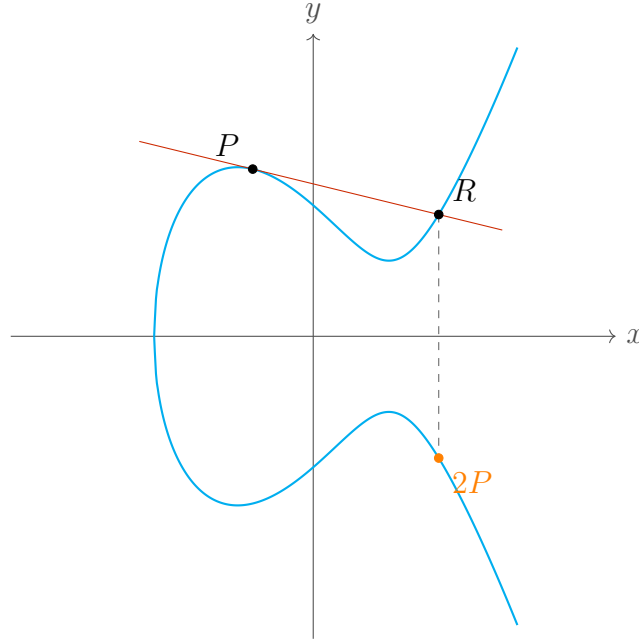


Figure 2.3: Somma di punti con la ascissa diversa

2.1.4 Moltiplicazione scalare

La moltiplicazione scalare consiste nella ripetizione della somma di un punto a se stesso, un numero arbitrario grande k volte, segnato come $S = kP$. Il problema si incontra quando, k , inevitabilmente, diventa molto grande; per evitare la difficoltà computazionale richiesta per sommare migliaia di volte un punto su un'equazione si usa un algoritmo chiamato **double-and-add**

2.1.5 Double-and-add

L'algoritmo *double and add* si basa sui principi della scomposizione binaria e sulle proprietà delle curve ellittiche.

Per capire in quale modo consideriamo lo scalare k nella sua forma binaria:

$$k = (k_{n-1}k_{n-2} \dots k_1k_0)_2 \quad (2.9)$$

dove k_i sono le cifre binarie (bit) di k . La rappresentazione binaria ci consente di esprimere k come:

$$k = \sum_{i=0}^{n-1} k_i \cdot 2^i \quad (2.10)$$

Utilizzando la stessa rappresentazione si può rappresentare kP come:

$$kP = \left(\sum_{i=0}^{n-1} k_i \cdot 2^i \right) P \quad (2.11)$$

Per la proprietà distributiva questa espressione può essere scritta come:

$$kP = \sum_{i=0}^{n-1} k_i \cdot (2^i P) \quad (2.12)$$

Il termine $2^i P$ rappresenta il punto P raddoppiato i volte. Perciò il problema si riduce ad una serie di raddoppi e addizioni di punti, il che rappresenta l'essenza dell'algoritmo.

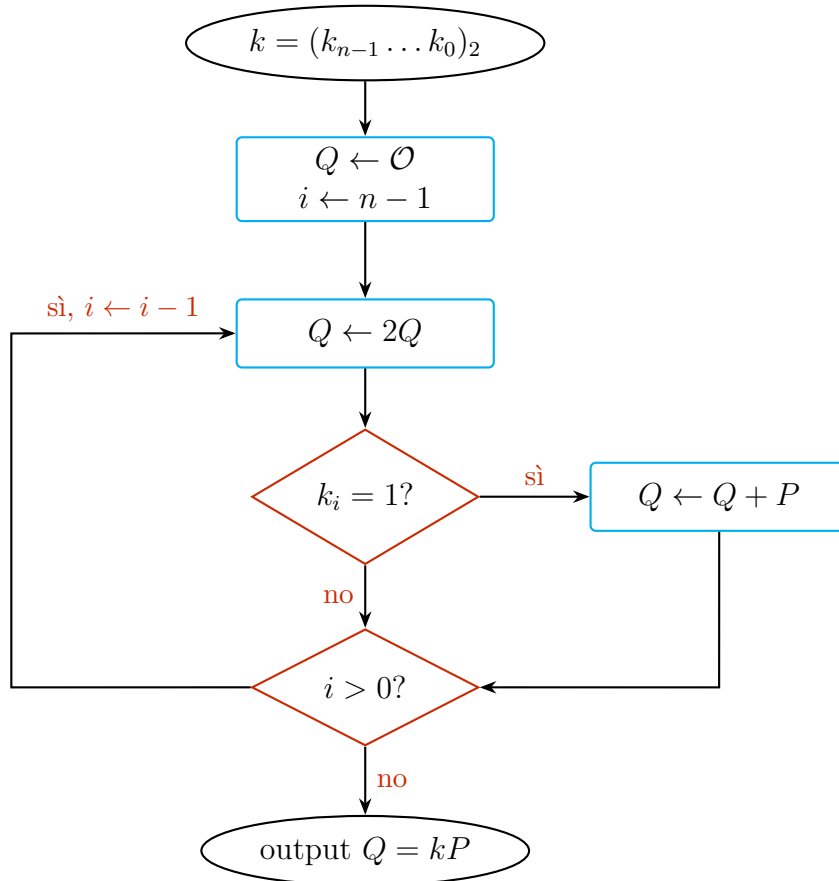


Figure 2.4: Algoritmo Double-and-add schematizzato

2.2 Curve su campo finito

Le curve ellittiche su $\mathbb{R}$ non sono di particolare interesse ai crittografi, si preferisce invece lavorare su curve ellittiche definite su campi finiti. Questa decisione permette di compiere operazioni veloci e precise dato dal fatto che tutte le operazioni riguardano coefficienti relativamente ridotti e interi, senza preoccuparsi di calcoli approssimati ed esageratamente lunghi.

In aggiunta operare in aritmetica modulare ci permette di sviluppare problemi che risultano computazionalmente difficili.

Per il resto di questa relazione si utilizzerà un campo $\mathbb{Z}_p$ con caratteristica $p > 3$ in modo da utilizzare l'equazione ellittica ridotta alla forma normale di Weierstrass.

Definizione 6. *Sia $p > 3$ un numero primo. La curva ellittica su $\mathbb{Z}_p$, denotata $E_p(a, b)$, è l'insieme dei punti che soddisfano l'equazione $y^2 = x^3 + ax + b \pmod{p}$ insieme al punto all'infinito.*

$$E_p(a, b) = \{(x, y) \in \mathbb{Z}_p^2 : y^2 = x^3 + ax + b \pmod{p}\} \cup \{\mathcal{O}\} \quad (2.13)$$

dove in aggiunta:

$$4a^3 + 27b^2 \neq 0 \quad (2.14)$$

Theorem 2.2.1 (Hasse). *Sia N il numero di punti di una curva ellittica E definita su $\mathbb{F}_q$. Allora*

$$|N - (q + 1)| \leq 2\sqrt{q} \quad (2.15)$$

Per la dimostrazione si veda [3]

I punti della curva $E_p(a, b)$ che rispettano queste due condizioni formano un gruppo abeliano finito rispetto all'operazione di addizione. [3]

2.2.1 Creazione di chiavi su $\mathbb{Z}_p$

Due utenti Angelica e Brian vogliono instaurare una comunicazione sicura, si concordano pubblicamente su un campo finito e su una curva ellittica definita su questo campo. Dopodiché scelgono un punto G appartenente alla curva di ordine n grande, in modo che $nG = \mathcal{O}$.

Angelica sceglie un intero positivo $n_a < n$ come chiave privata, genera una chiave pubblica $P_a = n_a G$ spedendola in chiaro a Brian.

Allo stesso modo Brian genera un punto casuale $P_b = n_b G$ che a sua volta viene spedito ad Angelica in chiaro.²

²I punti P_a e P_b rappresentano punti calcolati a partire da punti scelti casualmente, questi appartengono alla curva

Angelica riceve il punto P_b e calcola un nuovo punto $n_a P_b = n_a n_b G = S^3$, Brian compie la stessa operazione con n_b al posto di n_a in modo da ottenere lo stesso punto S

Per trasformare S in una chiave segreta condivisa occorre trasformarlo in un numero intero, convenzionalmente si considerano gli ultimi 256 bit dell'ascissa di S , ovvero:

$$k = x_s \pmod{2^{256}} \quad (2.16)$$

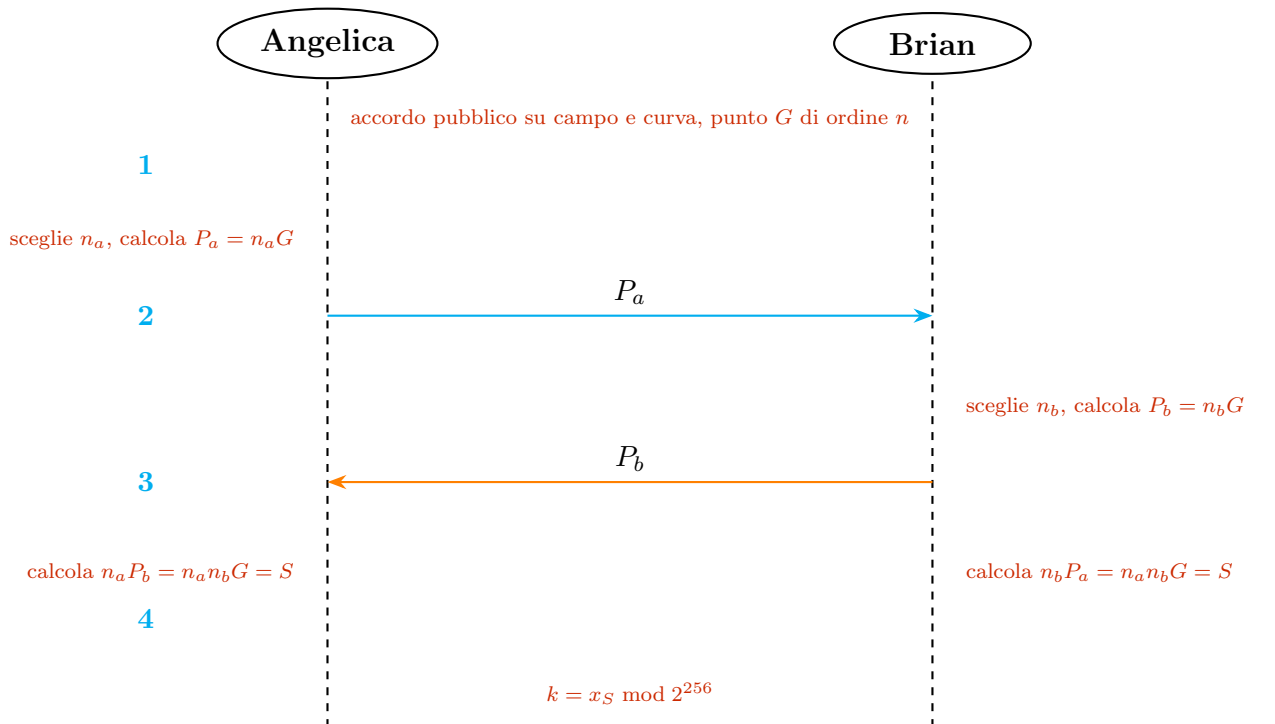


Figure 2.5: Creazione della chiave condivisa su $\mathbb{Z}_p$ tra Angelica e Brian.

2.2.2 Metodi a confronto

Mettendo a confronto i metodi crittografici visti fino ad ora si nota come le curve ellittiche utilizzino chiavi di dimensione notevolmente inferiore rispetto agli altri, il che le rende molto utili per dispositivi con risorse limitate, come smartcard, dispositivi IoT...

³ S è un nuovo punto appartenente alla curva

<i>TDEA, AES</i> (bit della chiave)	<i>RSA e DH</i> (bit del modulo)	<i>ECC</i> (bit dell'ordine)
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	512

2.2.3 Problema del logaritmo discreto

Il problema del logaritmo discreto ellittico (ECDLP) richiede di trovare un intero m tale che

$$mP = Q \quad (2.17)$$

per dati punti $P, Q \in E(\mathbb{F}_q)$. Questo problema viene considerato computazionalmente difficile: i migliori algoritmi noti, come Pollard's ρ e Baby-step Giant-step, hanno complessità $O(\sqrt{n})$, dove n è l'ordine del punto P . Inoltre, è stato dimostrato che nessun algoritmo generico può superare questo limite [3]

2.2.4 Algoritmi di attacco

Theorem 2.2.2 (Baby-step Giant-step – Shanks). *Dato un gruppo G , siano $x, y \in G$ e n l'ordine di x . Il seguente algoritmo risolve il PLD in $O(\sqrt{n})$ operazioni e $O(\sqrt{n})$ spazio [3]:*

1. Poniamo $N = \lceil \sqrt{n} \rceil$.
2. Calcoliamo i *babystep*:

$$x, x^2, x^3, \dots, x^N. \quad (2.18)$$

3. Poniamo $z = (x^N)^{-1}$ e calcoliamo i *giantstep*:

$$yz, yz^2, yz^3, \dots, yz^N. \quad (2.19)$$

4. Se esiste un'uguaglianza $x^i = yz^j$ tra le due liste, allora $y = x^{i+jN}$; altrimenti y non è una potenza di x .

La dimostrazione si può trovare in [3].

proposition 2.2.1 (Pohlig-Hellman attack). *L'algoritmo di Pohlig-Hellman [4] riduce l'ECDLP in un gruppo di ordine n a sottoproblemi nei sottogruppi di ordine primo di $\langle P \rangle$, utilizzando il Teorema Cinese del Resto per ricostruire la soluzione. La complessità dell'algoritmo dipende dal fattore primo più grande di n : se n è B -smooth⁴, l'attacco diventa efficiente. Per questa ragione, i parametri della curva devono essere scelti in modo che n sia divisibile da un primo grande.*

⁴con n -smooth si intende un numero i quali fattori primi sono tutti $\leq$ di n

Chapter 3

Protocolli basati su Curve Ellittiche

In questa sezione si affronteranno due protocolli applicati alle curve ellittiche: *firma digitale* e *cifratura integrata*

3.1 Firma digitale

Con lo sviluppo di internet si sono venuti a creare nuovi problemi, quelli che verranno discussi in questa sezione sono:

- **Identificazione:** un sistema di elaborazione deve essere in grado di accertare l'identità di un utente che richiede di accedere ai suoi servizi.
- **Autenticazione:** il destinatario di un messaggio deve essere in grado di accertare l'identità del mittente e l'integrità del crittogramma ricevuto.

Le firme digitali sono utilizzate per rappresentare una versione digitale della firma cartacea. Queste sono rappresentate da un numero basato su un segreto noto solamente all'utente che intende firmare (chiave privata), e in aggiunta sul contenuto del documento che viene firmato. Un aspetto fondamentale delle firme digitali è che ogni firma deve essere verificabile — se dovesse sorgere una controversia, un'entità terza imparziale deve essere in grado di risolvere la questione in modo equo senza la necessità di avere accesso alla chiave privata del mittente. Queste dispute possono sorgere nei seguenti casi:

- il mittente intende negare di aver firmato un messaggio da lui effettivamente firmato.

- il destinatario sostiene di aver ricevuto un messaggio diverso da quello inviatogli dal mittente.

La proprietà di sicurezza fondamentale di uno schema di firma digitale è l'*existential unforgeability under chosen-message attack*: un attaccante in grado di ottenere la firma dell'utente A per qualunque messaggio a sua scelta non deve essere in grado di falsificare la firma di A su un nuovo messaggio. [5]

3.1.1 Generazione delle chiavi per comunicare

prerequisiti

Per assicurarsi di utilizzare una curva ellittica che non sia vulnerabile ad attacchi crittografici noti è importante tenere in considerazione questi aspetti durante la scelta della curva:

1. un campo finito di ordine q , dove o $q = p^1$, un numero primo dispari, oppure $q = 2^m$
2. un'indicazione FR (*field representation*) della rappresentazione usata per gli elementi di $\mathbb{F}_q$
3. due elementi a, b che definiscono la curva ellittica E definita su $\mathbb{F}_q$ (es. $y^2 = x^3 + ax + b$ nel caso di $p > 3$, e $y^2 + xy = x^3 + ax^2 + b$ in caso $p = 2$)
4. due elementi del campo x_G e y_G su $\mathbb{F}_q$ i quali definiscono un punto finito $G = (x_G, y_G)$ di ordine primo in $E(\mathbb{F}_q)$
5. l'ordine n del punto G , con $n > 2^{160}$ e $n > 4\sqrt{q}$
6. il cofattore $h = \frac{\#E(\mathbb{F}_q)}{n}$

Per una spiegazione più dettagliata della motivazione dietro alla scelta di questi requisiti consultare [5]

Calcolo effettivo

Le coppie di chiavi di un dato utente A , *chiave pubblica* - *chiave privata* sono associate ad un insieme specifico dei parametri del dominio di una curva ellittica denominati $D = (q, FR, a, b, G, n, h)$, i valori che questi devono assumere

¹ p rappresenta la caratteristica del campo, importante quando si tratta campi come F_{2^m}

sono stati discussi precedentemente. È necessario che ogni utente si assicuri che i parametri del dominio siano validi prima della creazione delle chiavi [5]. Per generare una coppia di chiavi, *pubblica* - *privata*, ogni utente deve seguire tre passi:

1. Selezionare un numero casuale, o pseudo-casuale, d compreso nell'intervallo $[1, n - 1]$.
2. Calcolare $Q = dG$.
3. La chiave privata dell'utente è d ; la chiave pubblica è Q .

3.1.2 Verifica della chiave pubblica

Per validare una chiave pubblica Q ricevuta da un utente A , si possono usare uno di questi metodi:

1. A utilizza un algoritmo di validazione delle chiavi pubbliche, come descritto nell'algoritmo 3.1.2.
2. A genera Q utilizzando un sistema affidabile.
3. A riceve conferma da un'entità affidabile T la quale ha validato la chiave utilizzando un algoritmo simile a 3.1.2.
4. A riceve conferma da un'entità affidabile T che Q è stato generato utilizzando un sistema affidabile.

Algoritmo di validazione della chiave pubblica

INPUT: Una chiave pubblica $Q = (x_Q, y_Q)$ associata a parametri di dominio validi (q, FR, a, b, G, n, h) .

OUTPUT: Accettazione o rifiuto della validità di Q .

1. Controllare che $Q \neq \mathcal{O}$.
2. Controllare che x_Q e y_Q sono elementi correttamente rappresentati su $\mathbb{F}_q$ (es., interi compresi nell'intervallo $[0, p - 1]$ nel caso $q = p$, e stringhe di bit di lunghezza m nel caso di $q = 2^m$).
3. Controllare che Q appartiene alla curva ellittica definita da a e b .
4. Controllare che $nQ = \mathcal{O}$.
5. Se qualunque di questi controlli risultano falsi, allora Q non è valido; altrimenti Q è considerato valido.

3.1.3 Creazione di una firma digitale

Per firmare un messaggio m , un utente A con parametri di dominio $D = (q, FR, a, b, G, n, h)$ con chiavi (d, Q) deve seguire i seguenti passaggi:

1. Selezionare un numero casuale, o pseudocasuale, k all'interno dell'intervallo $1 \leq k \leq n - 1$.
2. Calcolare $kG = (x_1, y_1)$ e convertire x_1 in un intero $\bar{x}_1$.
3. Calcolare $r = x_1 \pmod{n}$. Se $r = 0$ allora tornare allo step 1.
4. Calcolare $k^{-1} \pmod{n}$.
5. Calcolare $\text{SHA-1}(m)^2$ e convertire la stringa di bit in un intero e .
6. Calcolare $s = k^{-1}(e + dr) \pmod{n}$. Se $s = 0$ allora tornare allo step 1.
7. La firma dell'utente A per il messaggio m è (r, s) .

²SHA-1 utilizzato per coerenza con la fonte citata; è oggi considerato obsoleto per la resistenza alle collisioni, e le implementazioni moderne di ECDSA utilizzano tipicamente SHA-256 o algoritmi successivi.

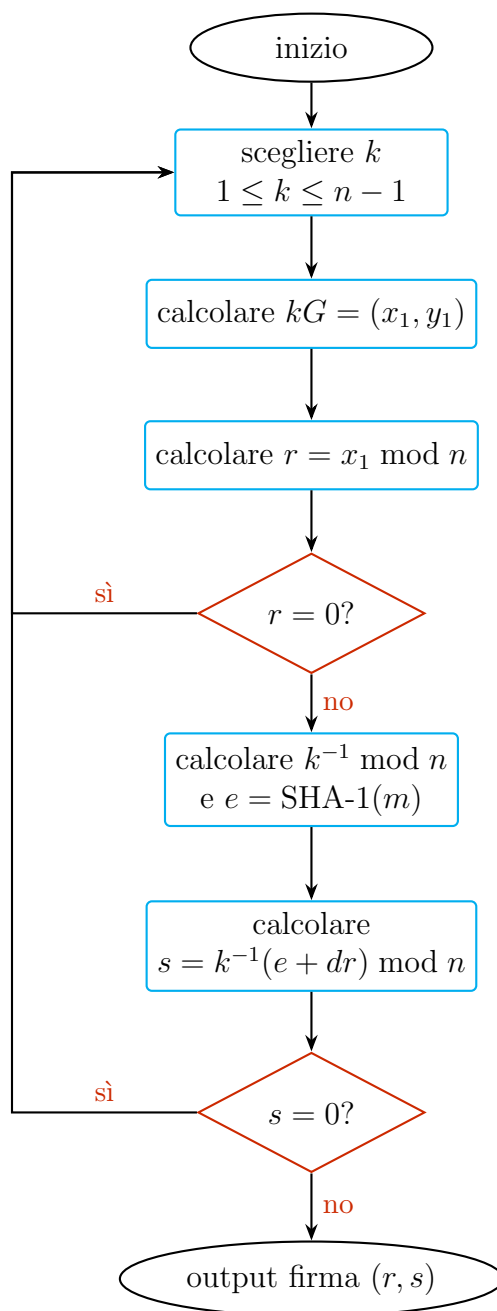


Figure 3.1: Flowchart per la creazione di una firma digitale ECDSA.

3.1.4 Potenziale vulnerabilità

Il segreto per messaggio k deve essere generato in modo indipendente per ciascun messaggio firmato. Altrimenti la chiave privata d può essere scop-

erta.³ Questa vulnerabilità è stata scoperta in un caso reale nel 2010, quando un gruppo di ricerca *fail0verflow* lo ha presentato alla conferenza 27C3 nel dicembre del 2010 svelando una grave vulnerabilità nei sistemi dell'azienda *Sony*.

Perché questo attacco possa essere messo in atto serve che l'utente che sta firmando i messaggi utilizzi la stessa k per molteplici messaggi, nel caso di Sony utilizzavano la stessa k come costante per ogni messaggio criptato inviato, se questo accade è possibile risalire alla chiave privata d in questa maniera:

0. Si suppone che Alice abbia usato la stessa k per due messaggi diversi m_1 e m_2 . Dato che k è uguale, allora lo è anche r ⁴. Le due firme sono:

$$s_1 \equiv k^{-1}(e_1 + dr) \pmod{n} \quad (3.1)$$

$$s_2 \equiv k^{-1}(e_2 + dr) \pmod{n} \quad (3.2)$$

1. Moltiplicare entrambi i lati per k :

$$ks_1 \equiv e_1 + dr \pmod{n} \quad (3.3)$$

$$ks_2 \equiv e_2 + dr \pmod{n} \quad (3.4)$$

2. Sottraiamo la seconda equazione dalla prima:

$$k(s_1 - s_2) \equiv e_1 - e_2 \pmod{n} \quad (3.5)$$

L'intuizione da notare è che scompare temporaneamente la chiave privata.

3. Se $s_1 \neq s_2$, si possono dividere entrambe le parti⁵:

$$k \equiv (s_1 - s_2)^{-1}(e_1 - e_2) \pmod{n} \quad (3.6)$$

4. Ora per trovare d basta invertire la formula originale:

$$d \equiv r^{-1}(ks_1 - e_1) \pmod{n} \quad (3.7)$$

Dato che s_1, s_2 si possono ricavare dalla firma e e_1, e_2 sono le hash dei messaggi si può ricavare tutto senza avere accesso alla chiave privata

³Se viene utilizzato un generatore di numeri casuali o pseudocasuali la possibilità di generare due numeri identici è trascurabile

⁴Questo è vero perché r rappresenta la coordinata x di $k \cdot G$

⁵Questa operazione è valida perché se $s_1 \equiv s_2$ è facile vedere che questo implica che $e_1 \equiv e_2$, il che è quasi impossibile utilizzando SHA-1 con $m_1 \neq m_2$, perciò è valido dare per scontato che $s_1 \neq s_2$

3.1.5 Verifica di una firma digitale

Per verificare la firma (r, s) dell'utente A per il messaggio m , l'utente B ottiene una copia autentica dei parametri di dominio di A : $D = (q, FR, a, b, n, h)$ e la chiave pubblica Q ad esso associata.

Una volta validati sia D che Q (per una dimostrazione formale consultare [5]), B deve procedere in questa maniera:

1. Verificare che r e s siano interi che appartengono all'intervallo $[1, n-1]$.
2. Calcolare $\text{SHA-1}(m)$ e convertire la stringa di bit in un intero e .
3. Calcolare $w = s^{-1} \pmod{n}$.
4. Calcolare $u_1 = ew \pmod{n}$ e $u_2 = rw \pmod{n}$.
5. Calcolare $X = u_1G + u_2Q$.
6. Se $X = \mathcal{O}$, allora rifiutare la firma. Altrimenti convertire la coordinata x di X in un intero $\bar{x}_1$ e calcolare $v = \bar{x}_1 \pmod{n}$.
7. Accettare la firma solo e soltanto se $v = r$.

3.2 Codifica di un messaggio

Una volta generate le chiavi pubbliche e private, e le necessarie firme digitali, i due utenti, A e B hanno bisogno di un protocollo in grado di cifrare questi messaggi per poi essere inviati.

Il protocollo usato per codificare messaggi attraverso curve ellittiche è denominato "*ECIES (elliptic curve integrated encryption scheme)*" che implementa le seguenti funzioni:

- Key Agreement (KA): Funzione usata per la generazione di un segreto condiviso tra due utenti.
- Key Derivation Function (KDF): Meccanismo che produce una coppia di chiavi derivate da parametri ideati per la generazione di tali chiavi e altri parametri opzionali.
- Encryption (ENC): Algoritmo di criptazione simmetrica.
- Message Authentication Code (MAC): Dati utilizzati in modo da autenticare messaggi.
- Hash (HASH): Funzione digest usata all'interno di KDF e MAC.

Per spiegare meglio il protocollo descritto sopra si propone un esempio. Si utilizzeranno come utenti *Alice*, la quale invierà il messaggio e *Bob* il quale riceverà il messaggio.

Consideriamo u, U e v, V rispettivamente le chiavi private e pubbliche dei due utenti. I passi che *Alice* dovrà seguire sono i seguenti:

1. Alice deve generare una coppia di chiavi effimera composta dall'elemento di campo finito u e dal punto della curva ellittica $U = u \cdot G$. Tale coppia deve essere generata pseudocasualmente.
2. Una volta generati Alice utilizzerà la funzione KA, per creare un valore segreto condiviso, il quale è il risultato della moltiplicazione scalare $u \cdot V$
3. Dopodiché Alice deve prendere il nuovo valore segreto condiviso ed usarlo, assieme ad altri valori opzionali, come input per la funzione KDF. L'output di questa funzione è la concatenazione della chiave di crittazione simmetrica, k_{ENC} , e la chiave MAC, k_{MAC}
4. Una volta creata k_{ENC} Alice userà l'algoritmo ENC, in modo da trasformare il messaggio in chiaro m in un messaggio criptato c .
5. Presi inoltre il crittogramma c , k_{MAC} e opzionalmente altri parametri, Alice deve usare la funzione MAC in modo da produrre un tag .
6. Infine, Alice prenderà U , tag e c e manderà il crittogramma $(U || tag || c)$ il quale consiste dei tre elementi concatenati, a Bob.

[14]

3.3 Decodifica di un messaggio

Per quanto riguarda la decodifica del messaggio ricevuto da Alice Bob deve seguire i seguenti passi:

1. Utilizzando la chiave pubblica U e la sua chiave privata v , Bob deve moltiplicarli assieme per ottenere lo stesso prodotto di Alice, $u \cdot V$
2. Usando come input il valore segreto condiviso $v \cdot U$ e gli stessi parametri opzionali che ha usato Alice, Bob deve produrre le stesse k_{ENC} e k_{MAC} utilizzando la stessa procedura con la funzione KDF.
3. Utilizzando la chiave k_{MAC} , il messaggio criptato c e gli stessi parametri opzionali usati da Alice, Bob deve calcolare il valore tag^* attraverso la

funzione MAC, per poi confrontarlo con il *tag* ricevuto da Alice. Se i due valori risultano diversi, Bob deve rifiutare il crittogramma a causa di un fallimento della procedura di verifica MAC.

4. Se invece il valore del tag è uguale, allora Bob può procedere a decifrare il messaggio *c* utilizzando l'algoritmo simmetrico ENC e la chiave k_{ENC} una volta decrittato Bob sarà in grado di leggere in chiaro il messaggio inviato da Alice.

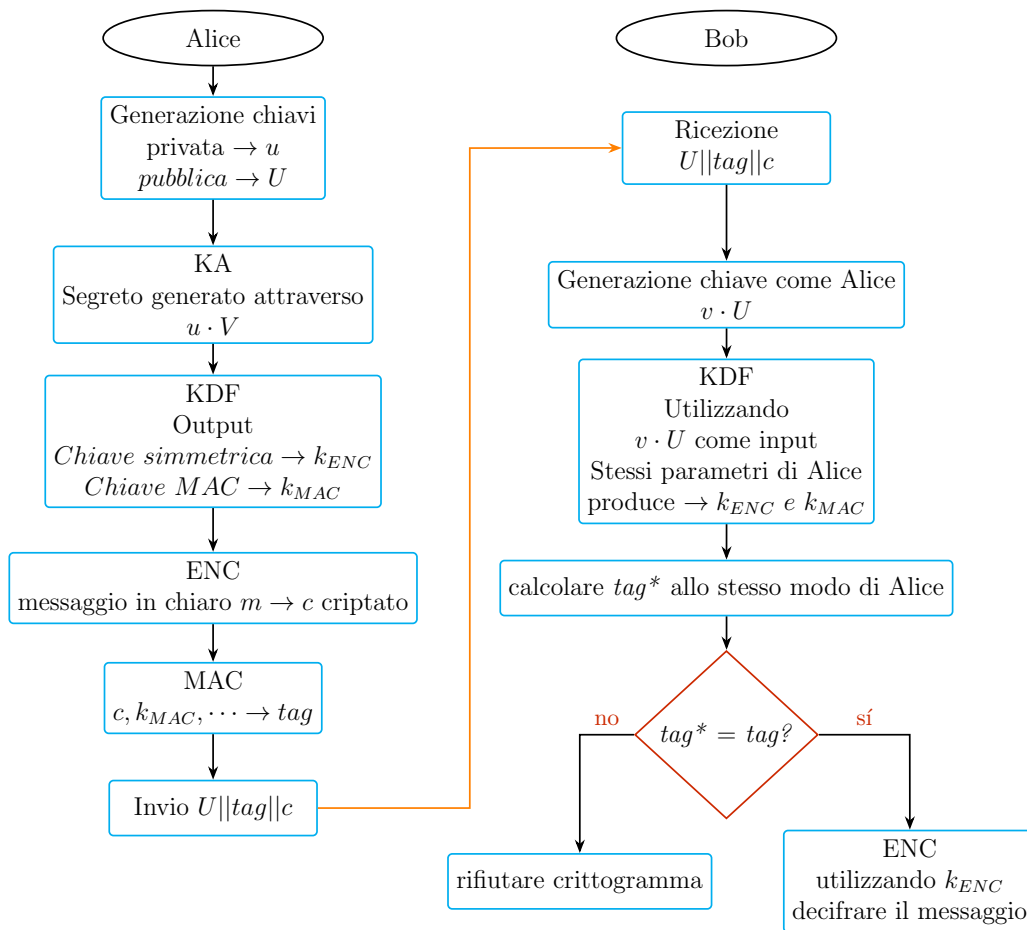


Figure 3.2: Codifica e decodifica di messaggi inviati tra Alice e Bob

Chapter 4

ECC nei Dispositivi IoT

4.1 Vincoli Hardware dei dispositivi IoT

In questa relazione verranno trattati due dispositivi IoT in particolare: *Arduino Uno* e *ESP-32*.

Le specifiche tecniche di questi due dispositivi sono le seguenti [6]:

Arduino Uno MCU

- 8 bit, 16MHz
- Memoria flash: 32 KB
- Memoria SRAM: 2 KB, EEPROM: 1 KB

ESP-32

- CPU: Xtensa dual-core 32-bit LX6 microprocessore, 160-240 MHz
- Memoria: 320 KiB RAM, 448 KiB ROM
- Wireless connectivity: Wi-Fi: 802.11 b/g/n + Bluetooth: v4.2 BR/EDR e BLE

4.2 Perché RSA è inadatto

Arduino Uno dispone di soli **2048 byte** di RAM totale. A differenza di quanto si potrebbe pensare sommando semplicemente le dimensioni di chiave, messaggio e risultato, la quantità di byte all'interno della RAM necessari per RSA risulta essere significativamente più alta, per tre motivi:

- La moltiplicazione di due numeri da 256 byte produce un prodotto intermedio, prima della riduzione modulo n , di fino a 512 byte;
- Un algoritmo di riduzione, come Montgomery, richiede ulteriori buffer temporanei;
- l'uso del CRT (Chinese Remainder Theorem), usato per rendere la decifratura sufficientemente veloce, richiede costanti precalcolate aggiuntive.

Questo è confermato da misurazioni dirette su hardware AVR: un'implementazione ottimizzata di RSA-2048 su un ATmega128 (un microcontrollore della stessa famiglia dell'ATmega328P trovato su Arduino Uno ma con il doppio della RAM) richiede 1332 byte di RAM per l'operazione a chiave pubblica e 1853 per quella a chiave privata con CRT. [15]

Considerando i 2048 byte disponibili su Arduino Uno, l'operazione a chiave pubblica occuperebbe da sola circa il **65%** della RAM totale, mentre l'operazione a chiave privata ne occuperebbe circa il **90%** senza considerare buffer e variabili necessarie al programma. Questo rende, nella pratica, la decifratura RSA-2048 non realizzabile su un dispositivo con le caratteristiche di Arduino Uno. Ci sono implementazioni di RSA su Arduino Uno come questo programma presente su gitea [↗](#).

Il problema con implementazioni come questa è che utilizzano una chiave da soli 32 bit anziché 2048, rendendo un attacco brute force semplice, quindi inutile la cifratura.

Per quanto riguarda l'ESP-32 che è in grado di supportare da un punto di vista di memoria le operazioni necessarie per utilizzare l'algoritmo di RSA, risulta inefficiente in confronto ad algoritmi ECC.

	Arduino Uno	ESP32
RAM disponibile	2 KB	320 KB
Chiave RSA-2048	impossibile	possibile ma lento
Chiave ECC-256	difficile	efficiente

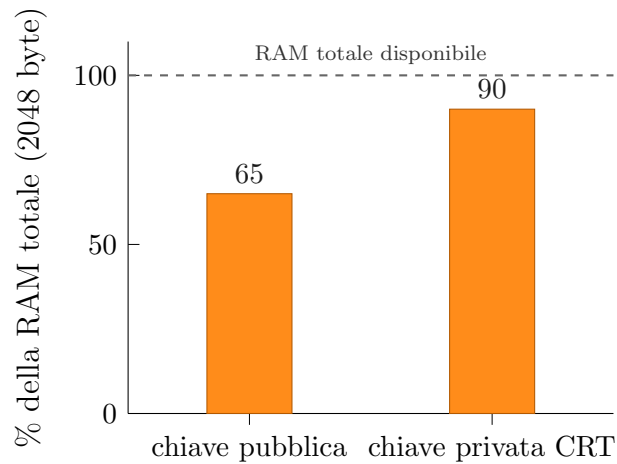


Figure 4.1: Occupazione della RAM di Arduino Uno per le operazioni RSA-2048 [15].

Studiando più nel dettaglio un confronto tra i due argomenti si può notare in quali aspetti algoritmi ECC siano più efficienti di algoritmi RSA, e invece in quali ambiti sia il contrario. I dati utilizzati in questa sezione si possono controllare in questo paper [7].

4.2.1 Requisiti di memoria

Considerando un livello di sicurezza pari a 128 e 160 si nota un vantaggio notevole da parte degli algoritmi ECC:

La grandezza delle chiavi risulta pari a 1024 e 2048 rispettivamente per quanto riguarda RSA, invece pari a 160 e 210 per quanto riguarda ECC; similmente si nota che i requisiti di memoria (in byte) risultano notevolmente superiori per RSA rispetto a ECC, con valori rispettivamente pari a 313, 621 e 125, 140.

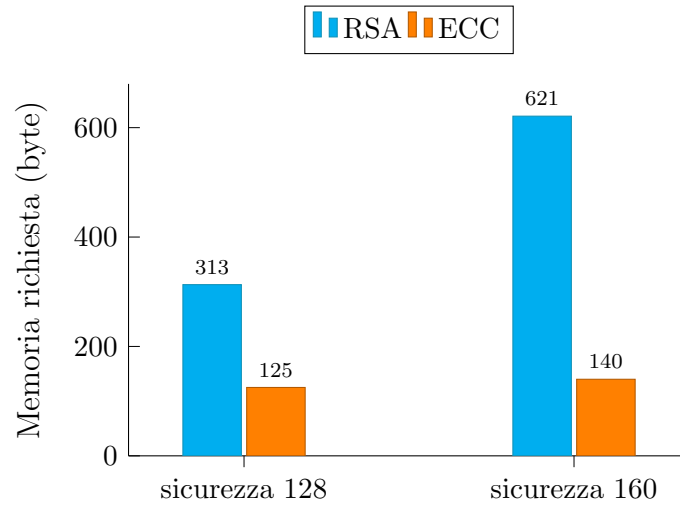


Figure 4.2: Requisiti di memoria (byte) per RSA ed ECC a due livelli di sicurezza [7].

4.2.2 Consumo energetico

Un altro aspetto molto importante per dispositivi IoT è il consumo energetico, un calcolatore come l'ESP-32 è spesso alimentato da una batteria, perciò ottimizzare il consumo energetico è importante per permettere al dispositivo di continuare ad operare senza il bisogno di cambiare la batteria. Vediamo dalle tabelle del paper citato sopra che a pari livelli di sicurezza l'algoritmo ECC comporta un consumo nettamente inferiore rispetto a quello di RSA, si passa da una differenza di $\sim 8mWh$ a livello di sicurezza pari a 80, ad una differenza di $\sim 40mWh$ a livello di sicurezza pari a 128.

4.2.3 Generazione delle Chiavi e Tempo di Esecuzione

A parità di sicurezza le chiavi generate dall'algoritmo RSA risultano costantemente più grandi e necessitano anche di più tempo per essere generate. Per quanto riguarda l'esecuzione dell'algoritmo invece RSA risulta essere più veloce quando si considera l'operazione con chiave pubblica, quando si considera l'operazione con chiave privata invece ECC risulta nettamente più veloce.

Date le restrizioni dei dispositivi considerati è chiaro che algoritmi ECC risultano più efficienti per quanto riguarda la cifratura e firma di messaggi, soprattutto quando si considerano dispositivi come *Smart Cards* o dispositivi che similmente necessitano di utilizzare una comunicazione sicura. Inoltre RSA

può essere più adatto ad applicazioni che richiedono la verifica di messaggi piuttosto che generazione di firme.

4.3 Protocolli di comunicazione

Nell'ambito dell'IoT è comune trovare numerosi dispositivi che comunicano allo stesso momento, alcuni dei quali connessi alla corrente altri invece che utilizzano una semplice batteria come alimentazione.

Per questi motivi il protocollo TLS usato come tale non risulta adatto in quanto richiede un canale di comunicazione affidabile e ordinato. TLS si affida allo strato TCP per ottenere queste caratteristiche di canale di comunicazione. Con questi requisiti, di conseguenza, si vengono a formare problemi di latenza, larghezza di banda, e risorse.

Per evitare questi problemi è stato sviluppato un nuovo protocollo basato su TLS: *DTLS*.

4.4 DTLS

Le differenze ad alto livello tra TLS e DTLS riguardano principalmente la "qualità" del canale di comunicazione; come annunciato precedentemente TLS utilizza un canale di comunicazione affidabile e ordinato, il che richiede molte risorse. Per evitare questo problema DTLS utilizza un canale affidabile e ordinato soltanto durante il protocollo di *handshaking*¹, il resto della comunicazione non presenta questo tipo di accertamento, il che implica che informazioni si possano perdere durante la comunicazione. Per evitare la perdita totale di dati il server che implementa DTLS invia un messaggio *ACK* per segnalare le informazioni ricevute, una volta ricevuto il messaggio il cliente può controllare se manca dell'informazione e di conseguenza inviare solo i dati mancanti. Questi accertamenti permettono a dispositivi che implementano il protocollo DTLS a utilizzare meno risorse rispetto a TLS e di lasciare i canali di comunicazione più liberi.

4.4.1 Scambio di chiavi su DTLS

Come già discusso nelle sezioni [3.1 2.2.1] il server autentica i parametri effimeri tramite firma *ECDSA* e le due parti calcolano il valore segreto tramite

¹handshaking inteso come la fase iniziale dove il cliente e il server si accertano dell'identità l'uno dell'altro

ECDH. Nello specifico nel protocollo DTLS questo meccanismo si inserisce nella fase di *handshake* in questo modo:

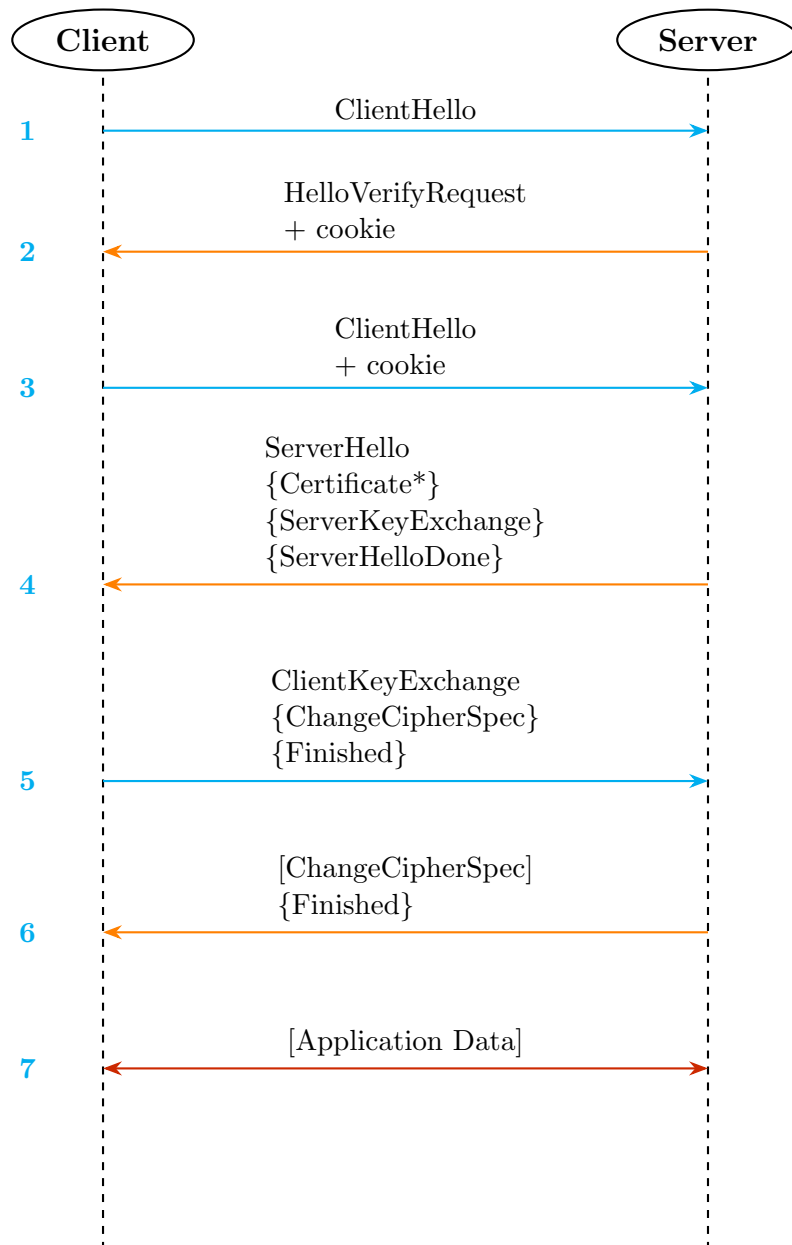


Figure 4.3: DTLS handshake

Il formato del messaggio *ClientHello* deve essere della forma:

```
enum {
    elliptic_curves(10),
    ec_point_formats(11)
} ExtensionType;
```

[8]

Dove `elliptic_curves` indica le curve ellittiche supportate dal cliente. `ec_point_formats` invece indica i punti che il cliente può analizzare.

Una volta accordati la curva e i punti appartenenti a questa da utilizzare (vedere [8] per dettagli implementativi) il server invierà un *ServerHello* che include un messaggio denominato *Server Key Exchange*. Questo messaggio include la chiave pubblica *ECDH* effimera assieme ai parametri della curva, i quali sono firmati tramite *ECDSA* utilizzando la chiave privata del certificato su un hash che include i valori random di entrambe le parti.

Visivamente si può rappresentare in questo modo:

```
SHA(ClientHello.random + ServerHello.random + params)
```

Una volta ricevuto questo messaggio il cliente recupera i parametri di dominio della curva ellittica e la chiave pubblica *ECDH* effimera del server.

Successivamente il cliente invia un *Client Key Exchange* che include una chiave pubblica *ECDH* che corrisponde ai parametri ricevuti dal server. A quel punto il server controlla che la chiave appartenga alla stessa curva della sua chiave *ECDH*.

Entrambe le parti calcolano indipendentemente il premaster secret tramite moltiplicazione scalare (come discusso in 2.2.1), dal quale vengono derivate le chiavi simmetriche utilizzate per cifrare i dati applicativi nel passo 7.

4.5 Curva P-256

Prima di parlare di come dispositivi IoT utilizzano e sfruttano le curve ellittiche è buono introdurre una delle curve più popolari: *P-256*, anche chiamata *secp256r1* oppure *prime256v1*. Questa curva, standardizzata dal NIST, presenta come parametri consigliati i seguenti:

- p = FFFFFFFF 00000001 00000000 00000000 00000000 FFFFFFFF FFFFFFFF FFFFFFFF
- a = FFFFFFFF 00000001 00000000 00000000 00000000 FFFFFFFF FFFFFFFF FFFFFFFFC
- b = 5AC635D8 AA3A93E7 B3EBBD55 769886BC 651D06B0 CC53B0F6 3BCE3C3E 27D2604B

- $S = \text{C49D3608 86E70493 6A6678E1 139D26B7 819F7E90}$
- $G = \text{03 6B17D1F2 E12C4247 F8BCE6E5 63A440F2 77037D81 2DEB33A0}$
 F4A13945 D898C296
- $n = \text{FFFFFFFF 00000000 FFFFFFFFF FFFFFFFFF BCE6FAAD A7179E84 F3B9CAC2}$
 FC632551
- $h = 01$

P-256 (secp256r1) è raccomandata da NIST in [10] ed è obbligatoria per l'interoperabilità in TLS secondo [8], il che ne spiega l'adozione massiccia. Questi parametri assieme ad altri parametri consigliati per diverse curve si possono trovare leggendo [9]

4.6 ESP32 mbedTLS

All'interno del framework ufficiale di Espressif per ESP32 è presente una libreria crittografica chiamata *mbedTLS*, la quale fornisce un'implementazione di TLS 1.2/1.3, supporto a DTLS, primitive crittografiche e generazione di numeri casuali; per una descrizione dettagliata consultare [11].

Come supplemento a questa libreria il dispositivo ESP32 presenta anche un acceleratore hardware usato per velocizzare operazioni AES, assieme a operazioni SHA. Per specifiche tecniche su queste componenti consultare [12].

Queste due caratteristiche dell'ESP32 permettono di compiere calcoli su ECC su un microcontrollore, altrimenti queste operazioni risulterebbero troppo lente, rendendole inefficienti per dispositivi come questo.

4.7 Note aggiuntive

Il protocollo *Matter* utilizza la curva P-256 durante la comunicazione tra dispositivi IoT [13]; questo standard è stato adottato da aziende rilevanti quali *Google*, *Apple* e *Amazon*.

Chapter 5

Conclusioni

Con lo sviluppo della tecnologia e dei dispositivi utilizzati sul posto di lavoro si è arrivati a riuscire a creare dispositivi di dimensione ridotta ma i quali riescono comunque a completare operazioni complesse e diversificate.

Una conseguenza di questa nuova popolarità è la necessità di comunicare in maniera sicura nonostante le risorse relativamente limitate di questi dispositivi. Grazie agli sviluppi e ai progressi nell'ambito crittografico separato da quello di IoT si è visto che i metodi di comunicazione più efficienti per dispositivi simili a quelli usati nell'IoT sono i protocolli asimmetrici. Il problema quindi si è trasformato nel cercare di sviluppare algoritmi crittografici in grado di operare efficacemente su dispositivi con risorse limitate.

Algoritmi quali *RSA* o *Diffie-Hellman*, i quali si basano sulla difficoltà computazionale di fattorizzare grandi numeri nonostante molto efficaci su dispositivi in confronto potenti, risultano inadatti in quanto la dimensione elevata delle chiavi comporta un severo rallentamento delle operazioni rendendole praticamente inutili quando applicate a comunicazioni in tempo reale tra dispositivi.

La soluzione a questo problema la si trova nella forma delle *curve ellittiche* le quali si basano sulla difficoltà di risolvere il problema del logaritmo discreto; questi algoritmi sono stati ideati con l'obiettivo di raggiungere livelli di sicurezza comparabili ma utilizzando chiavi di dimensione notevolmente ridotta in modo da rendere possibile l'utilizzo di questi sui dispositivi nominati sopra

Alcune delle applicazioni più note di questi algoritmi sono la firma digitale, *ECDH*, *DTLS*, *Matter*, utilizzati su dispositivi come *Arduino UNO* e *ESP-32* come esempi menzionati sopra.

Come visto dal confronto dell'algoritmo *RSA* e *ECC* in condizioni di sicurezza equivalente, la dimensione della chiave usata in *ECC* è notevolmente inferiore a quella utilizzata in *RSA*, la quale nella pratica non risulta utilizzabile in

ambito IoT.

Il percorso qui delineato, dalla teoria dei campi finiti fino all'implementazione pratica su dispositivi come Arduino Uno ed ESP-32, mostra come un problema apparentemente astratto — la struttura algebrica delle curve ellittiche — trovi una delle sue applicazioni più rilevanti proprio dove le risorse computazionali sono più scarse.

Bibliography

- [1] Sabrina De Capitani di Vimercati, Pierangela Samarati *Encyclopedia of Cryptography, Security and Privacy*, Springer, 2025
- [2] L. Margara, *Crittografia su Curve Ellittiche*, Unibo, 2026.
- [3] J. H. Silverman, *The Arithmetic of Elliptic Curves*, 2nd ed., Springer, 2009.
- [4] Darrel Hankerson, Alfred Menezes, Scott Vanstone *Guide to Elliptic Curve Cryptography*, Springer, 2004
- [5] Don Johnson, Alfred Menezes, Scott Vanstone *The Elliptic Curve Digital Signature Algorithm (ECDSA)*, Springer, 2001
- [6] Alessandro Ricci, *Embedded systems and IoT*, Unibo, 2025
- [7] Z. Vahdati, S. M. Yasin, A. Ghasempour, M. Salehi, *Comparison of ECC and RSA Algorithms in IoT Devices*, Journal of Theoretical and Applied Information Technology
- [8] Y. Nir, S. Josefsson, M. Pegourie-Gonnard, *Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS) Versions 1.2 and Earlier*, RFC 8422, IETF, August 2018.
- [9] Certicom Research,
SEC 2: Recommended Elliptic Curve Domain Parameters, Version 2.0, SECG, January 2010. <https://www.secg.org/sec2-v2.pdf>
- [10] National Institute of Standards and Technology (NIST), *Digital Signature Standard (DSS)*, FIPS PUB 186-4, July 2013.
<https://csrc.nist.gov/publications/detail/fips/186/4/final>
- [11] Espressif Systems, *mbedTLS – ESP-IDF Programming Guide*,
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/>

- api-reference/protocols/mbedtls.html, consultato il 2 Giugno, 2026.
- [12] Espressif Systems, *ESP32 Technical Reference Manual*, Version 5.4, https://documentation.espressif.com/esp32_technical_reference_manual_en.pdf, consultato il 3 Giugno, 2026.
- [13] Connectivity Standards Alliance, *Matter 1.2 Core Specification*, CSA, 2023.
<https://csa-iot.org/wp-content/uploads/2023/10/Matter-1.2-Core-Specification.pdf>
- [14] Gayoso Martínez, V. and Hernández Encinas, L. and Sánchez Ávila, C. *A Survey of the Elliptic Curve Integrated Encryption Scheme* Journal of Computer Science and Engineering, 2010
- [15] N. Gura, A. Patel, A. Wander, H. Eberle, S. Chang Shantz, *Comparing Elliptic Curve Cryptography and RSA on 8-bit CPUs*, Cryptographic Hardware and Embedded Systems – CHES 2004, Lecture Notes in Computer Science, vol. 3156, Springer, 2004.